



DSAI352

RC-Bowling-Tracker

Contents

1	Introduction	3
2	System Architecture	3
2.1	High-Level Layer Diagram	3
2.2	Processing Pipeline	3
2.3	Technology Stack	4
3	Model Details	4
3.1	Architecture and Training	4
3.2	Classes	5
3.3	Inference on Device	5
4	Pin Tracking and Fall Detection	5
4.1	Matching Algorithm	5
4.2	Fall Detection	6
4.3	Track Maintenance	6
5	Video Processing and Rendering	6
5.1	Decode-Infer-Annotate-Encode Loop	6
5.2	EGL Surface Encoder Input	7
5.3	Resource Cleanup	7
6	React Native Bridge	8
7	Application Screens	8
8	Key Design Decisions	9
9	Output	9
9.1	Annotated Video	9
9.2	Structured Result Data	9
10	Future Work: iOS Layer	10
10.1	Motivation	10
10.2	Architecture Overview	10
10.3	Model Conversion for iOS	11
10.4	Video Processing Pipeline on iOS	11
10.5	React Native Bridge on iOS	12
10.6	Key Differences and Challenges	12

1 Introduction

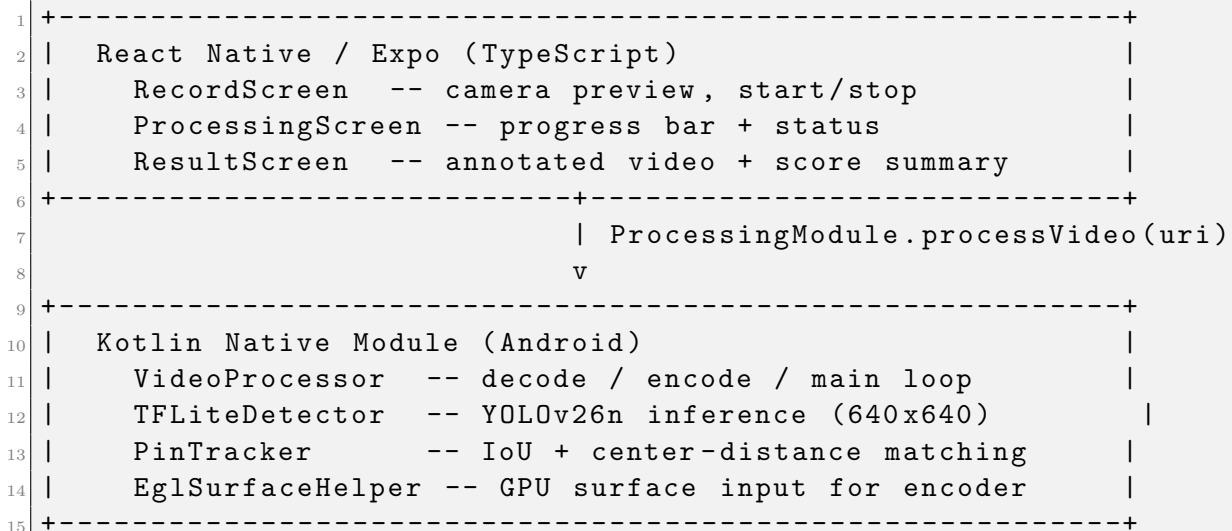
RC Bowling Tracker is an Android mobile application that records a toy RC car bowling setup, runs fully offline YOLOv26 object detection on every frame, tracks pin falls and car trajectory, and produces an annotated output video with fall order, path overlay, and scoring—all on-device without any network dependency.

The project was developed for the DSAI352 Bonus Project at Zewail City. The core requirement is to build a mobile application that captures a live demonstration video of an RC car knocking down bowling pins, processes the video entirely on-device using a custom-trained or fine-tuned object detection model, detects and orders pin falls, optionally tracks the car's trajectory, and produces an annotated output video with scoring overlays.

2 System Architecture

The system is divided into two major layers: a **React Native / Expo** front-end written in TypeScript, and a **Kotlin native module** that handles the entire computer vision pipeline.

2.1 High-Level Layer Diagram



2.2 Processing Pipeline

The end-to-end pipeline proceeds as follows:

1. The user records a video via the camera screen (**expo-camera**).
2. The raw video is saved to local storage.

3. The React Native orchestration layer passes the video URI to the `Kotlin ProcessingModule` over the React Native bridge.
4. The native module copies the video to a temporary file and initialises the `TFLiteDetector` and `PinTracker`.
5. `VideoProcessor` decodes frames via `MediaCodec`, runs TFLite YOLOv26n inference on each frame, updates the tracker, draws annotations on an `Android Canvas`, and pushes the annotated bitmap to the hardware encoder through an EGL/OpenGL ES surface.
6. The encoded H.264 output is muxed into an MP4 via `MediaMuxer` and saved to the device gallery.
7. Structured result metadata (elapsed time, pin events, car path) is returned to the JavaScript layer for display on the result screen.

2.3 Technology Stack

Layer	Technology
UI	React Native, Expo, TypeScript, Expo Router
Build	EAS Development Build (custom native modules)
Camera	<code>expo-camera</code>
Video Playback	<code>expo-video</code>
Native Code	Kotlin, React Native Bridge (<code>ReactContextBaseJavaModule</code>)
Inference	TensorFlow Lite Android (CPU, 4 threads)
Model	Fine-tuned YOLOv26n $\rightarrow$ TFLite (unquantized float32)
Video I/O	Android <code>MediaCodec</code> , <code>MediaMuxer</code> , <code>MediaExtractor</code>
Encoder Input	OpenGL ES 2.0 surface (EGL + GLES20)
Tracking	IoU + center-distance matching (rule-based)
Rendering	Android <code>Canvas</code> / <code>Bitmap</code> drawing

Table 1: Technology stack summary.

3 Model Details

3.1 Architecture and Training

The detector is a **YOLOv26n** (nano) model fine-tuned via transfer learning on a custom toy bowling dataset. The model was trained using the Ultralytics YOLO framework and exported to TensorFlow Lite format.

- **Input size:** 640×640 pixels.
- **Output:** $[1, 300, 6]$ — up to 300 detections, each $[x_1, y_1, x_2, y_2, \text{confidence}, \text{class_id}]$.
- **NMS:** Built into the exported TFLite model (post-processing fused at export time).
- **Format:** Float32 (unquantized, full precision), approximately 10 MB.

3.2 Classes

The model detects four classes:

ID	Class	Annotation Color	Confidence Threshold
0	ball	Yellow (gold)	0.35
1	car	Sky blue	0.35
2	fallen-pins	Orange-red + glow	0.35
3	standing-pins	Lime green	0.35

Table 2: Detection classes, annotation colours, and confidence thresholds.

3.3 Inference on Device

The `TFLiteDetector` class loads the model from Android assets at initialisation. Each frame is resized to 640×640 , normalised to $[0, 1]$, and packed into a direct `ByteBuffer` in CHW RGB format. The interpreter runs with 4 CPU threads.

The detector automatically inspects the output tensor shapes at init time and supports two output formats:

1. **NMS'd output** $[1, N, 6]$: coordinates are treated as normalised and scaled to the original frame dimensions.
2. **Raw YOLO output** $[1, 8, 8400]$ (transposed) or $[1, 8400, 8]$: centre-form boxes are converted to corner-form, and a greedy per-class NMS with an IoU threshold of 0.5 is applied in post-processing.

This dual-path design ensures compatibility with different TFLite export configurations of the same YOLOv26 model.

4 Pin Tracking and Fall Detection

The `PinTracker` mirrors the logic from the reference Colab notebook (`CV_New_method.ipynb`).

4.1 Matching Algorithm

On each frame, every pin detection is compared against all existing tracks. A detection is a candidate match for a track if either:

- The Euclidean distance between the bounding-box centres is less than 70 px, **or**
- The IoU between the two boxes exceeds 0.20.

The match score is computed as:

$$\text{score} = \text{IoU} - \frac{\text{distance}}{1000}$$

The detection is assigned to the track with the highest score. If no existing track matches, a new track is created with a fresh ID.

4.2 Fall Detection

A pin is marked as *fallen* the instant a `standing-pins` $\rightarrow$ `fallen-pins` class transition is observed for the corresponding track. No multi-frame confirmation window is used; this matches the reference notebook implementation and avoids delayed or missed fall events.

At the moment of transition:

- The fall order is assigned as a monotonically increasing integer.
- The fall time is computed as `frameIndex/fps` (seconds).
- The presentation timestamp is recorded by `VideoProcessor` for accurate event timing.

4.3 Track Maintenance

Tracks that are not matched for more than 20 consecutive frames are pruned. This prevents ghost tracks from accumulating when pins leave the field of view.

Parameter	Value
Match distance threshold	70 px
Match IoU threshold	0.20
Match score	$\text{IoU} - (\text{dist}/1000)$
Fall detection	Instant on <code>standing</code> $\rightarrow$ <code>fallen</code> transition
Track pruning	Remove after > 20 missing frames

Table 3: PinTracker parameters.

5 Video Processing and Rendering

5.1 Decode–Infer–Annotate–Encode Loop

`VideoProcessor` implements a synchronous pipeline that runs on a background thread:

1. **Decode:** A hardware `MediaCodec` decoder extracts raw frames. The decoder attempts `getOutputImage()` for YUV 4:2:0 Image access; if this returns `null` (common on many hardware decoders), it falls back to raw `ByteBuffer` YUV conversion.

2. **Infer:** Each decoded bitmap is passed to `TFLiteDetector.detect()`.
3. **Track:** Pin detections are fed to `PinTracker.update()`; car detections are appended to the path log.
4. **Annotate:** A new bitmap is drawn on an Android `Canvas` with:
 - Colour-coded bounding boxes (gold for ball, sky blue for car, lime green for standing pins, orange-red with orange glow for fallen pins).
 - Pin IDs and fall-order labels.
 - Car path line (purple with black shadow underlay).
 - Standing/fallen pin count panel (top-left).
 - Fall history timeline panel (top-right).
5. **Encode:** The annotated bitmap is uploaded to a GPU texture via `GLUtils.texImage2D()` and rendered onto the encoder’s input `Surface` through an EGL fullscreen quad. The presentation timestamp is set via `eglPresentationTimeANDROID()`.

5.2 EGL Surface Encoder Input

Instead of feeding raw pixel buffers to the encoder (which causes stride/format mismatch corruption across different Android hardware encoders), the system uses an **OpenGL ES 2.0 surface input** approach:

1. An EGL context and window surface are created from the encoder’s `InputSurface`.
2. A simple vertex + fragment shader pair renders a textured fullscreen quad.
3. Each annotated bitmap is uploaded as a GL texture, drawn, and then the surface is swapped with the correct presentation timestamp.

This completely eliminates row-shift corruption caused by stride mismatches—the GPU handles all pixel format conversion internally.

5.3 Resource Cleanup

All codec resources are managed within a **try-finally** block with strict ordering:

1. EGL helper and input surface release.
2. Encoder stop and release.
3. Decoder stop and release.
4. Extractor release.
5. Muxer stop (only if started and has data) and release **last**.

This ordering prevents the “Failed to stop the muxer” error that occurs when Android’s `MediaMuxer.release()` internally calls `stop()` on a half-written file.

6 React Native Bridge

The `ProcessingModule` Kotlin class extends `ReactContextBaseJavaModule` and exposes two methods to JavaScript:

- `isNativeAvailable()`: returns `true` to confirm that the native module is loaded.
- `processVideo(inputVideoUri, enableCarPath)`: launches video processing on a background thread and resolves the promise with a `WritableMap` containing the output video URI, elapsed time, pin events array, and car path array.

Progress updates are emitted as React Native device events (`ProcessingProgress`) containing the current stage, percentage, and human-readable message. The UI subscribes to these events to display a real-time progress bar on the processing screen.

7 Application Screens

The UI is built with Expo Router and consists of the following screens:

`RecordScreen`

Camera preview with start/stop recording controls. Uses `expo-camera` for video capture.

`ProcessingScreen`

Displays a progress bar and status messages received from the native module's progress events.

`ResultScreen`

Plays the annotated output video (`expo-video`) and displays a score summary: elapsed time, total pins knocked down, and a table of pin-fall events with timestamps and hit order.

`DebugScreen`

Development-time diagnostics: model load status, inference timing, detection counts, and error logs.

8 Key Design Decisions

Decision	Rationale
Surface encoder input (EGL/GLES20)	Eliminates stride/format mismatch corruption across all Android hardware encoders.
Direct ByteBuffer YUV decode fallback	Hardware <code>getOutputImage()</code> returns <code>null</code> on many devices; raw buffer fallback is universal.
Monotonic timestamp enforcement	Non-monotonic PTS corrupts the native MP4 muxer (<code>IllegalStateException</code>).
<code>try-finally</code> resource cleanup	Prevents “Failed to stop the muxer” errors from leaked <code>MediaCodec</code> / <code>MediaMuxer</code> instances.
Instant fall detection (no multi-frame confirm)	Matches notebook reference implementation; avoids delayed/missed fall events.
Unquantized float32 model	Better detection accuracy for the small toy bowling objects vs. quantized variant.
Dual output format support	Handles both NMS’d $[1, N, 6]$ and raw transposed $[1, 8, 8400]$ TFLite exports transparently.

Table 4: Key design decisions and their rationale.

9 Output

The application produces two outputs:

9.1 Annotated Video

An annotated MP4 video saved to the device gallery, containing:

- Colour-coded bounding boxes for all detected objects.
- Pin IDs and fall-order labels with timestamps.
- Car trajectory path (purple line with black shadow).
- Standing/fallen pin count panel (top-left, semi-transparent dark background with lime accent bar).
- Fall History timeline panel (top-right, semi-transparent dark background with orange-red accent bar).

9.2 Structured Result Data

A JSON object returned to the JavaScript layer:

```
1 {
2   "outputVideoUri": "content://...",
3   "elapsedMs": 8500,
4   "pinsKnockedDown": 4,
5   "pinEvents": [
6     { "pinTrackId": 2, "order": 1, "timeMs": 1200 },
7     { "pinTrackId": 5, "order": 2, "timeMs": 1800 },
8     ...
9   ],
10  "carPath": [
11    { "timeMs": 0, "x": 120, "y": 600 },
12    ...
13  ]
14 }
```

Listing 1: Structured result JSON returned to the JS layer.

10 Future Work: iOS Layer

The current implementation targets Android exclusively. This section describes the planned architecture for an iOS native layer that would bring the same on-device inference and video processing capabilities to iPhones and iPads.

10.1 Motivation

Although the Android layer satisfies all project requirements, iOS support would:

- Expand the user base to iOS devices, which represent a large share of the mobile market.
- Leverage Apple’s hardware-accelerated Neural Engine and GPU cores for potentially faster inference.
- Demonstrate true cross-platform capability of the React Native / Expo architecture.

10.2 Architecture Overview

The iOS layer would mirror the Android architecture with platform-native equivalents:

Component	Android (Current)	iOS (Planned)
Native Language	Kotlin	Swift
Bridge	ReactContextBaseJavaModule	RCTBridgeModule or Expo Modules Swift API
Video Decode	MediaCodec + MediaExtractor	AVAssetReader + AVAssetReaderTrackOutput
Video Encode	MediaCodec + MediaMuxer	AVAssetWriter + AVAssetWriterInput
Encoder Surface	EGL + OpenGL ES 2.0 surface	AVAssetWriterInputPixelBufferAdaptor
Inference Runtime	TensorFlow Lite Android	Core ML (.mlmodel) or TensorFlow Lite iOS
Frame Drawing	Android Canvas / Bitmap	Core Graphics (CGContext) / UIImage
Gallery Save	MediaStore content provider	PHPhotoLibrary

Table 5: Platform equivalences between the Android and planned iOS layers.

10.3 Model Conversion for iOS

Two inference runtime options are available on iOS:

1. **Core ML** (recommended): Convert the trained YOLOv26n model to the `.mlmodel` format using the `coremltools` Python package. Core ML automatically leverages the Apple Neural Engine (ANE) on A12+ and M-series chips, yielding significantly faster inference than CPU-only execution. The conversion pipeline would be:

```
1 YOLOv26n (PyTorch) -> ONNX -> Core ML (.mlmodel)
```

Listing 2: Core ML model conversion pipeline.

Ultralytics provides a built-in `model.export(format="coreml")` command that handles this in one step.

2. **TensorFlow Lite iOS**: Use the same `.tflite` model file as Android with the TensorFlow Lite Swift/Obj-C runtime. This maximises model parity but forfeits ANE acceleration (TFLite on iOS supports CPU and GPU delegate only, not the Neural Engine).

The Core ML path is recommended because it provides hardware acceleration with zero additional configuration and integrates natively with the Apple toolchain.

10.4 Video Processing Pipeline on iOS

The iOS video processing module (`VideoProcessorIOS.swift`) would implement the same decode–infer–annotate–encode loop:

1. **Decode**: `AVAssetReader` with a `kCVPixelFormatType_32BGRA` output setting extracts frames as `CVPixelBuffer` objects. Unlike Android, iOS decoders produce pixel buffers with well-defined strides, so no YUV conversion fallback is needed.

2. **Infer:** Each `CVPixelBuffer` is resized to 640×640 (via `vImage` or Core Graphics) and fed to the Core ML model using `VNCoreMLRequest` (Vision framework integration) or direct `MLModel.prediction()` calls.
3. **Track:** The same tracking algorithm (IoU + centre-distance matching) would be re-implemented in Swift. The `PinTracker` logic is purely arithmetic and translates directly.
4. **Annotate:** Core Graphics (`CGContext`) draws bounding boxes, labels, path lines, and the HUD panels onto a `CGImage/UIImage`.
5. **Encode:** `AVAssetWriter` with an `H.264 AVAssetWriterInput` encodes annotated frames. A `AVAssetWriterInputPixelBufferAdaptor` manages the pixel buffer pool and presentation timestamps—no manual EGL/OpenGL setup is needed, as Apple’s API handles surface management internally.

10.5 React Native Bridge on iOS

The iOS native module would expose the same JavaScript API:

```
1 // Same JS call, different native implementation
2 const result = await ProcessingModule.processVideo(uri, true);
```

Listing 3: JavaScript call — identical API on both platforms.

Using the Expo Modules API (Swift), the bridge definition would be:

```
1 import ExpoModulesCore
2
3 public class ProcessingModule: Module {
4     public func definition() -> ModuleDefinition {
5         Name("ProcessingModule")
6
7         AsyncFunction("processVideo") {
8             (uri: String, enableCarPath: Bool) -> [String: Any] in
9             return try await VideoProcessorIOS.process(
10                 inputUri: uri, enableCarPath: enableCarPath
11             )
12         }
13
14         Events("ProcessingProgress")
15     }
16 }
```

Listing 4: Expo Modules Swift bridge definition for iOS.

Because the React Native / TypeScript layer is already platform-agnostic, no changes to the UI code would be required—only the native module registration.

10.6 Key Differences and Challenges

1. **Development environment:** iOS builds require macOS with Xcode; the current development is on Arch Linux. A macOS machine or CI service (e.g., EAS Build cloud) would be needed.

2. **Pixel format handling:** iOS decoders output BGRA pixel buffers with consistent strides, eliminating the YUV conversion and stride mismatch issues encountered on Android. This simplifies the decode stage significantly.
3. **Neural Engine acceleration:** Core ML on A12+ chips can run the YOLOv26n model on the 16-core Neural Engine, potentially achieving sub-10 ms per-frame inference—much faster than the CPU-only TFLite path on Android.
4. **Video encoding:** `AVAssetWriter` provides a higher-level API than Android’s `MediaCodec/MediaMuxer` combination. There is no need for manual EGL surface management or monotonic timestamp enforcement—Apple’s writer handles these details.
5. **Privacy and permissions:** iOS requires explicit `NSCameraUsageDescription` and `NSPhotoLibraryUsageDescription` entries in `Info.plist`, plus runtime permission prompts managed through the Expo permissions API.

The iOS layer would be implemented in the following order:

1. Convert the YOLOv26n model to Core ML format and validate inference output parity with the TFLite model.
2. Implement the Swift `VideoProcessorIOS` with decode–encode loop (without inference) to verify video I/O.
3. Integrate Core ML inference and validate detection outputs on sample frames.
4. Port the `PinTracker` to Swift and validate fall-order assignment.
5. Implement the annotation renderer using Core Graphics.
6. Wire up the Expo Module bridge and test end-to-end from the React Native UI.
7. Perform device testing on multiple iPhone models (iPhone 12 and newer recommended for Neural Engine performance).

11 Conclusion

RC Bowling Tracker demonstrates a complete on-device computer vision pipeline—from video capture through object detection, multi-object tracking, and annotated video rendering—running entirely on a mobile phone without any server dependency.

The key technical contributions of the project include:

- A robust dual-format TFLite output parser that handles both NMS-processed and raw YOLO outputs.
- An EGL surface-based encoder input strategy that eliminates cross-device stride corruption.
- A deterministic resource cleanup protocol that prevents muxer failures across all Android hardware.
- A clean separation between the React Native UI and the native processing pipeline, enabling straightforward future extension to iOS as detailed in Section 10.

The architecture is designed to be modular: the same TypeScript UI code drives both the Android and future iOS implementations, with only the native video processing and inference layers needing platform-specific code.